Course Code: CSA1741 Course Name: ARTIFICIAL INTELLIGENCE
Slot: B

ASSIGNMENT 4 – CO4

Nature of Assignment: Individual Duration: 1 Week Total Marks: 50

Student Name: Niranjna devi S I Reg. No:192519100

CO4 Coverage: Learning from observation – Inductive learning – Decision trees – Explanation-based learning – Statistical learning methods – Learning with complete data and hidden variables – Neural networks – Kernel machines – Reinforcement learning.

Purpose of the Assignment

This assignment assesses whether you can think through a learning algorithm **before** coding it and then engineer, test, break and defend it **after** coding it. Each task therefore contains a **[Design]** component (completed on paper, in pseudocode and by manual trace, before any program is written) and an **[Implement]** component (code, outputs and performance metrics as proof).

You are expected to follow the progression:

Interpret the Problem → Identify the State or Decision Rule → Design the Logic → Write Pseudocode → Trace Manually → Implement → Instrument → Challenge → Refine

TASK 1 – Pseudocode Review: Diagnose and Repair a Poor Learning Protocol (5 Marks)

A junior engineer submits the following pseudocode for building a student at-risk predictor:

```
ALGORITHM TrainAndTest(Data)
BEGIN
  Read Data
  Train a suitable model on Data
  Test the model on Data
  PRINT accuracy
END
```

Task 1.1 – Diagnose the Protocol [Design] | 2 Marks

Identify at least five deficiencies in this algorithm (for example, evaluation on training data, no held-out test set, no validation for choosing hyper-parameters, accuracy alone on imbalanced data, no baseline, no control of randomness, ambiguous statements such as “a suitable model”). Explain why each one can mislead an engineer.

Task 1.2 – Repair the Protocol [Design] | 2 Marks

Write corrected pseudocode that includes data splitting, k-fold cross-validation for model selection, a held-out test set used once, the metrics to be computed and a majority-class baseline. Do not write "apply a suitable model".

Task 1.3 – Demonstrate the Defect [Implement] | 1 Mark

On a dataset of your choice, show with code and output how much the training-set accuracy overestimates the cross-validated and held-out accuracy.

Evidence Expected: Defect list with reasons, repaired pseudocode, code, output and a numeric demonstration of the optimistic estimate.

TASK 2 – From Requirement to Pseudocode to Working Model: Decision Trees (9 Marks)

A college wants an early-warning system that predicts whether a student will pass or fail from attendance, assignment status and internal marks. You are not provided with the algorithm.

Task 2.1 – Model the Requirement [Design] | 1 Mark

Before writing pseudocode, identify the input, output, objective, hypothesis space, constraints, the decision made at each node and the information required to choose an attribute.

Task 2.2 – Formulate a Splitting Hypothesis [Design] | 1 Mark

State, in one precise sentence, the attribute-selection rule you propose and briefly justify why it can contribute to a small, accurate tree.

Task 2.3 – Design the Algorithm [Design] | 2 Marks

Write complete pseudocode for recursive tree construction. It must clearly expose the stopping conditions (no examples, all examples of one class, no attributes remaining), the attribute-selection computation, the partitioning of examples and the construction of subtrees.

Task 2.4 – Desk-Check [Design] | 1 Mark

Manually trace your algorithm using:

No.	Attendance	Assignments	Internal Marks	Result
1	High	Done	Good	Pass
2	High	Done	Poor	Pass
3	High	Missed	Good	Pass
4	High	Missed	Poor	Fail
5	Low	Done	Good	Pass
6	Low	Done	Poor	Fail

No.	Attendance	Assignments	Internal Marks	Result
7	Low	Missed	Good	Fail
8	Low	Missed	Poor	Fail

Show the entropy and information-gain calculations that determine the root and the important decisions, not merely the final tree.

Task 2.5 – Implement and Verify [Implement] | 2 Marks

Implement the learner from scratch (entropy and Gini criteria) on a public dataset of your choice. Verify it against scikit-learn on the same split. Report accuracy, precision, recall, F1-score and the confusion matrix, with code and output as proof.

Task 2.6 – Overfitting and Pruning [Implement] | 1 Mark

Grow the full tree and then limit depth or minimum samples (or use reduced-error pruning). Record depth, number of leaves and train, validation and test performance, and explain the effect.

Task 2.7 – Challenge Your Design | 1 Mark

Change one value in the desk-check table. Before tracing again, predict whether the root attribute changes. Then verify by tracing and by running the code. Add an identifier-like attribute with unique values to demonstrate the bias of information gain and describe a repair.

Evidence Expected: Problem model, splitting rule, complete pseudocode, gain-based trace, scikit-learn comparison, metrics, pruning table and modified-case reasoning.

TASK 3 – Pseudocode Forensics: Hidden Variables and the EM Algorithm (6 Marks)

The following pseudocode is intended to fit a mixture of *K* one-dimensional Gaussians to unlabelled data.

```
ALGORITHM GaussianMixtureEM(X, K, T)
INPUT: X – n data points, K – components, T – iterations
OUTPUT: weights pi[k], means mu[k], variances var[k]
BEGIN
  Initialise pi[k], mu[k], var[k] for k = 1..K
  FOR t <- 1 TO T DO
    FOR i <- 1 TO n DO
      FOR k <- 1 TO K DO
        r[i][k] <- pi[k] * Normal(X[i] | mu[k], var[k])
      END FOR
    END FOR
    FOR k <- 1 TO K DO
      Nk <- SUM over i of r[i][k]
      mu[k] <- SUM over i of r[i][k] * X[i] / Nk
      var[k] <- SUM over i of r[i][k] * (X[i] - mu[k])^2 / Nk
      pi[k] <- Nk / n
    END FOR
  END FOR
```

```
  RETURN pi, mu, var
END
```

The pseudocode contains a logical defect.

Task 3.1 – Locate the Defect [Design] | 1 Mark

Identify the exact step that is incomplete and explain which quantity no longer has its required meaning.

Task 3.2 – Construct a Failing Input and Trace [Design] | 1 Mark

Design your own small dataset (at least 4 points, K = 2) and initial parameters. Trace one iteration to show where the first incorrect value appears.

Task 3.3 – Repair [Design] | 1 Mark

Correct only the required logic without rewriting the whole algorithm.

Task 3.4 – Implement Faulty and Repaired Versions [Implement] | 2 Marks

Implement both versions. Plot the log-likelihood per iteration and the sum of the mixing weights for each version. Show that the repaired version never decreases the log-likelihood and report results over at least 20 random initializations.

Task 3.5 – Regression Check | 1 Mark

Provide one input on which the original already worked, and demonstrate that the repair preserves correct behavior.

Evidence Expected: Exact defect, student-designed failing case, state trace, minimal correction, code, log-likelihood plots and regression evidence.

TASK 4 – From Recurrence to Executable Logic: Neural Network Back-Propagation (9 Marks)

Task 4.1 – Define the State [Design] | 1 Mark

Define precisely, for a network with layers $l = 1 \ldots L$, the quantities z^(l), a^(l) and the error term δ^(l), including their dimensions.

Task 4.2 – Develop the Forward and Backward Recurrences [Design] | 2 Marks

Develop the forward recurrence and the backward recurrence for the error terms using the chain rule. Explain the meaning of each term, the boundary condition at the output layer and why the error terms are computed from the last layer to the first.

Task 4.3 – Translate into Pseudocode [Design] | 2 Marks

Write complete pseudocode for mini-batch training: initialisation, forward pass, loss, backward pass, weight update and stopping criterion.

Task 4.4 – Desk-Check [Design] | 1 Mark

A 2-2-1 sigmoid network has input x = (1, 0), target y = 1, hidden weights (0.5, −0.4) and (0.3, 0.8), output weights (0.6, −0.2), all biases 0, squared-error loss and learning rate 0.5. Trace one complete forward and backward pass and the weight update.

Task 4.5 – Implement and Verify [Implement] | 2 Marks

Implement the network in NumPy. Verify your analytic gradients against numerical gradients and report the maximum relative error. Show that a single-layer perceptron fails on XOR while your network succeeds. Reproduce your Task 4.4 trace in code.

Task 4.6 – Stress the Design [Implement] | 1 Mark

On the 8 × 8 digits dataset, vary hidden units {2, 8, 32, 64}, learning rate {0.001, 0.01, 0.1, 1.0} and activation {sigmoid, ReLU}. Predict first, then record loss curves, accuracy, macro-F1 and the confusion matrix. Provoke one failure (divergence or vanishing gradient) and diagnose it using per-layer gradient norms.

Evidence Expected: State definitions, recurrences with reasoning, complete pseudocode, hand trace, gradient-check output, XOR demonstration, metric tables and failure diagnosis.

TASK 5 – Same Dataset, Different Learning Reasoning (8 Marks)

A smart-grid operator has labelled records of substation states and wants to detect hazardous overload conditions.

Task 5.1 – Statistical Learning with Complete Data: Naive Bayes | 3 Marks

- **[Design]** Write pseudocode for parameter estimation and classification using maximum-likelihood counts with Laplace smoothing. Show on a three-example toy table why a zero count breaks classification. *(1 Mark)*
- **[Implement]** Implement from scratch on a text or tabular dataset with $\alpha \in \{0, 0.1, 1, 10\}$. Report accuracy, precision, recall, F1 and log-loss. *(2 Marks)*

Task 5.2 – Change the Assumption: Kernel Machines | 3 Marks

The hazardous and normal states are not linearly separable.

- **[Design]** Identify which assumption of your earlier models becomes unsafe, and explain the role of the margin, support vectors and kernel function. *(1 Mark)*
- **[Implement]** Compare a linear SVM and an RBF SVM, searching C and γ with 5-fold cross-validation. Record support-vector count, accuracy, F1 and training time and plot decision boundaries. Compare with Naive Bayes and your Task 2 tree on an identical split. *(2 Marks)*

Task 5.3 – Change the Requirement: Explanation-Based Learning | 2 Marks

Engineers now want the system to learn a reusable rule from a **single** explained hazardous case.

- **[Design]** Write a domain theory of at least 6 first-order rules, construct the explanation tree for one example and generalise it into an operational rule. *(1 Mark)*

- **[Implement]** Test the learned rule on 20 cases you design and compare inference steps with proving each case from the domain theory. State what was learned that the domain theory did not already entail. *(1 Mark)*

Evidence Expected: Pseudocode, toy-table reasoning, metric tables, decision-boundary plots, three-model comparison, explanation tree, learned rule and inference-step comparison.

TASK 6 – DESIGN CHALLENGE: Reinforcement Learning for an Energy-Aware Smart Controller (13 Marks)

A university lecture hall is fitted with a smart air-conditioning controller. Every 15 minutes the controller must choose between **Off**, **Low**, **High**. Occupancy changes through the day, outdoor temperature varies and electricity price changes by time slot.

The objective is to **reduce energy cost while keeping the room comfortable whenever it is occupied.**

You are not told which learning algorithm to use. You must develop the solution from the problem requirements.

Stage 1 – Interpret the Problem | 2 Marks

Identify the input (what the controller observes), output (action), objective, constraints, decisions to be made, and how an episode begins and ends. Do not select an algorithm yet.

Stage 2 – Propose the Strategy | 2 Marks

Consider alternatives such as a fixed-rule thermostat, supervised imitation of past operators, Q-learning, SARSA and a model-based approach. Justify your choice from the characteristics of the problem, not merely because it works on a test run.

Stage 3 – Identify the Critical Information | 2 Marks

Define the state representation (and why it is Markov enough), the action set, the reward function (energy cost and comfort violation), the exploration strategy and the discount factor.

Stage 4 – Design Complete Pseudocode | 3 Marks

Write complete, language-independent pseudocode conforming to the SIMATS Engineering Standard for Algorithmic Pseudocode. It must show initialisation, the interaction loop, action selection with exploration, the value update, termination and extraction of the final policy. Another programmer should be able to implement it without clarification.

Stage 5 – Desk-Check | 1 Mark

On a toy environment of three states and two actions that you define, manually execute at least four updates showing state → action → reward → next state → updated value.

Stage 6 – Implement, Instrument and Attack Your Own Design | 3 Marks

Implement a simulator and your learner with code and output as proof. Record reward per episode (smoothed), energy used (kWh), number of comfort violations, episodes to convergence and results over at least 20 seeds, compared with the fixed-rule baseline. Then attack your design: construct a situation (for example, an unusual price pattern, or a reward that can be exploited by switching off before occupants arrive) in which a locally attractive action produces a poor overall result. Predict the behaviour before running, then report what happened and which assumption or reward term failed.

Evidence Expected: Precise problem interpretation, reasoned strategy selection, state/reward design, complete pseudocode, manual trace, learning curves, baseline comparison and a student-designed adversarial test.

ASSESSMENT DISTRIBUTION – ASSIGNMENT 4

Task	Competency Assessed	Marks
Task 1	Diagnose and repair a learning protocol	5
Task 2	Design and implement decision-tree learning	9
Task 3	Debug hidden-variable learning (EM) with traces and tests	6
Task 4	Design, implement and verify neural-network learning	9
Task 5	Compare statistical, kernel and explanation-based learning	8
Task 6	Independently design and prototype a reinforcement-learning solution	13
	TOTAL	**50**

SUBMISSION REQUIREMENTS

For each applicable task, submit:

Problem Interpretation → Inputs/Outputs → Objective/Constraints → State or Decision Rule → Pseudocode → Manual Trace → Source Code → Test Input → Expected Behaviour → Actual Output → Performance Metrics → Boundary/Failure Test → Interpretation

Screenshots containing only a final output such as “Accuracy = 0.97” will not constitute sufficient evidence. Metrics must be reported on held-out data, never on training data alone.

Important Instructions

1. This is an individual assignment.
2. Complete the **[Design]** components (reasoning, pseudocode and manual traces) before writing any program for that task.

3. Programming code in Python or any other language will not be accepted as pseudocode. All pseudocode must follow the SIMATS Engineering – Handbook for Algorithmic Pseudocode.
4. Avoid statements such as “Train a suitable model”, “Tune the parameters appropriately”, “Choose the best features” or “Apply machine learning”. Such statements hide the algorithmic decision and will be considered incomplete.
5. Library implementations (scikit-learn, etc.) may be used only for verification or comparison where the task says so.
6. A correct final answer without the required reasoning, pseudocode, trace or experimental evidence will not receive full credit.
7. The Day-4 Design Challenge (Task 6) is compulsory.

SOFTWARE ENGINEERING EXPECTATIONS

Your implementation will also be examined for meaningful names, modular design, separation of algorithm logic from input/output, handling of boundary conditions, clear assumptions, fixed random seeds and reproducible test cases. A program that produces the expected output but whose logic cannot be explained or validated will not receive full credit.

IMPORTANT RULE

DO NOT STOP WHEN THE PROGRAM RUNS. Establish: *Does it produce the correct result? → When does it fail? → Why does it work? → What happens when the requirement changes?*

ENGINEERING WORKFLOW: DESIGN → IMPLEMENT → INSTRUMENT → TEST → BREAK → DEBUG → VALIDATE → DEFEND

SOLUTIONS

Task 1 – Learning Protocol (Short Answer)

1.1 Problems in the given protocol:

1. Training data is used for testing.
2. No separate testing dataset.
3. No validation or cross-validation.
4. Accuracy alone is not sufficient.
5. No baseline model is used.
6. Randomness is not controlled.
7. Model selection criteria are unclear.

1.2 Improved learning protocol:

1. Split the dataset into 70% training and 30% testing.
2. Use 5-fold cross-validation on training data.
3. Train different models and compare their performance.
4. Select the best model based on F1-score.
5. Evaluate it using test data.

1.3 Python code:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
data = load_breast_cancer()
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
```

)

model = DecisionTreeClassifier(random_state=42)

scores = cross_val_score(model, X_train, y_train, cv=5)

model.fit(X_train, y_train)

predictions = model.predict(X_test)

print("CV Accuracy:", scores.mean())

print("Test Accuracy:", accuracy_score(y_test, predictions))

Task 2 – Decision Tree Learning

2.1 Problem definition:

- **Input:** Attendance, assignment submission and internal marks.
- **Output:** Student result – Pass or Fail.
- **Goal:** Predict student performance.
- **Model:** Decision Tree.
- **Method:** Information Gain.

2.2 Attribute selection:

Calculate Information Gain for each attribute and select the attribute with the highest Information Gain as the root node. Repeat this process for the remaining branches.

2.3 Decision tree algorithm:

1. Start with the training dataset.

2. Calculate entropy.

3. Calculate Information Gain for each attribute.

4. Select the attribute with highest Information Gain.

5. Split the dataset based on that attribute.

6. Repeat until all records are classified.

7. Return the decision tree.

2.4 Entropy and Information Gain:

$$Entropy(S) = -\sum p_i \log_2(p_i)$$

$$IG(S, A) = Entropy(S) - Entropy(S|A)$$

For the given dataset:

- Total records = 8
- Pass = 4, Fail = 4
- Initial entropy = 1
- Information Gain for Attendance = 0.189
- Information Gain for Assignment = 0.189
- Information Gain for Internal Marks = 0.189

All three attributes have equal Information Gain. Attendance is selected as the root using a tie-break.

2.5 Python implementation:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
X = [[90,1,80], [85,1,75], [70,0,60],
    [60,1,50], [40,1,45], [30,0,35],
    [50,0,40], [20,0,25]]
y = [1,1,1,0,1,0,0,0]
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.25, random_state=42
)
model = DecisionTreeClassifier(criterion="entropy")
```

```
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
```

2.6 Effect of tree depth:

```
for d in [1, 2, 3, 5, None]:
    model = DecisionTreeClassifier(max_depth=d)
    model.fit(X_train, y_train)
    print(d, model.score(X_test, y_test))
```

A shallow tree may underfit, while a very deep tree may overfit. Pruning helps improve generalisation.

2.7 Handling noisy data:

If one student's result changes from Fail to Pass, the entropy and Information Gain values may change. To reduce noise and overfitting:

- Remove irrelevant attributes.
- Apply pruning.
- Use cross-validation.
- Collect more reliable training data.

Conclusion: Decision trees use Information Gain to select attributes and classify students into Pass or Fail.

Task 3 – Expectation Maximisation (EM) Algorithm

3.1 Identify the error:

The responsibility values are not normalised. Each observation's responsibilities must sum to 1.

3.2 Correct formula:

$$r_{ik} = \frac{\pi_k N(x_i|\mu_k, \sigma_k^2)}{\sum_j \pi_j N(x_i|\mu_j, \sigma_j^2)}$$

3.3 Correct algorithm:

1. Initialise means, variances and weights.
2. Calculate responsibilities.
3. Normalise responsibilities.
4. Update means, variances and weights.
5. Repeat until convergence.

3.4 Python code:

```
import numpy as np
from sklearn.mixture import GaussianMixture
X = np.array([[1], [2], [5], [6]])
model = GaussianMixture(n_components=2, random_state=42)
model.fit(X)
print("Means:", model.means_)
print("Weights:", model.weights_)
print("Responsibilities:")
print(model.predict_proba(X))
```

Conclusion: Normalizing responsibilities ensures correct probability values and improves EM algorithm performance.

Task 4 – Neural Network Backpropagation

4.1 Basic concepts:

- Forward propagation calculates the network output.
- Loss function measures prediction error.
- Backpropagation calculates gradients.
- Gradient descent updates weights and biases.

4.2 Backpropagation equations:

Output layer:

$$\delta_o = (\hat{y} - y)\sigma'(z_o)$$

Hidden layer:

$$\delta_h = (W^T \delta_o)\sigma'(z_h)$$

Weight update:

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W}$$

4.3 Algorithm:

1. Initialise weights and biases.
2. Perform forward propagation.
3. Calculate loss.
4. Calculate output error.
5. Propagate error backwards.
6. Update weights and biases.
7. Repeat until the loss decreases.

4.4 Python implementation:

```
import numpy as np
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
data = load_iris()
```

```
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42
)
model = MLPClassifier(
    hidden_layer_sizes=(10,),
    max_iter=1000,
    random_state=42
)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred))
```

Conclusion: Backpropagation helps neural networks learn by calculating errors and updating weights to improve predictions.

TASK 5 – Same Dataset, Different Learning Reasoning (8 Marks)

Task 5.1 – Statistical Learning with Complete Data: Naive Bayes

Design:

Naive Bayes is a statistical learning algorithm based on Bayes' theorem. It assumes that input features are conditionally independent given the class.

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)}$$

Laplace smoothing:

$$P(x_i|C) = \frac{Count(x_i, C) + \alpha}{Count(C) + \alpha V}$$

Pseudocode:

BEGIN

Calculate class frequencies.

Calculate prior probabilities.

Calculate feature probabilities.

Apply Laplace smoothing.

For each new record:

Calculate posterior probability for each class.

Predict the class with maximum probability.

RETURN predicted class.

END

Toy example:

Condition	Hazardous	Normal
High temperature	2	0
Normal temperature	0	1

Without smoothing, a zero probability makes the entire classification probability zero when that feature occurs. Laplace smoothing prevents this problem by adding a small value to each count.

Implementation:

```
from sklearn.naive_bayes import MultinomialNB

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.metrics import (

    accuracy_score, precision_score,

    recall_score, f1_score, log_loss

)

data = load_iris()

X, y = data.data, data.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42,
    stratify=y
)
for alpha in [0, 0.1, 1, 10]:
    model = MultinomialNB(alpha=alpha)
    model.fit(X_train, y_train)

    pred = model.predict(X_test)
    prob = model.predict_proba(X_test)
    print("Alpha:", alpha)
    print("Accuracy:", accuracy_score(y_test, pred))
    print("Precision:", precision_score(
        y_test, pred, average="macro", zero_division=0))
    print("Recall:", recall_score(
        y_test, pred, average="macro", zero_division=0))
    print("F1:", f1_score(
        y_test, pred, average="macro"))
    print("Log Loss:", log_loss(y_test, prob))
```

Task 5.2 – Kernel Machines

When hazardous and normal states are not linearly separable, a simple linear decision boundary may not classify them correctly.

An SVM solves this problem using:

- **Margin:** Distance between the decision boundary and nearest data points.
- **Support vectors:** Data points that determine the decision boundary.
- **Kernel function:** Maps data into a space where non-linear patterns can be separated.

Implementation:

```
from sklearn.svm import SVC

from sklearn.model_selection import GridSearchCV

parameters = {

    "C": [0.1, 1, 10],

    "gamma": [0.01, 0.1, 1]

}

for kernel in ["linear", "rbf"]:

    search = GridSearchCV(

        SVC(kernel=kernel),

        parameters,

        cv=5,

        scoring="f1_macro"

    )

    search.fit(X_train, y_train)

    model = search.best_estimator_

    pred = model.predict(X_test)

    print("\nKernel:", kernel)

    print("Best Parameters:", search.best_params_)

    print("Support Vectors:", len(model.support_))

    print("Accuracy:", accuracy_score(y_test, pred))

    print("F1:", f1_score(y_test, pred, average="macro"))
```

Comparison: Naive Bayes uses probability-based classification, decision trees use attribute-based splitting, and SVM uses maximum-margin classification. RBF SVM can handle non-linear decision boundaries.

Task 5.3 – Explanation-Based Learning

Explanation-Based Learning (EBL) uses prior domain knowledge to explain a training example and generalise the explanation into a reusable rule.

Domain theory – Six rules:

1. High current indicates increased electrical load.
2. High temperature indicates equipment heating.
3. Excessive load increases overheating risk.
4. Overheating can damage substation equipment.
5. High current and high temperature together indicate a hazardous condition.
6. A hazardous condition requires an immediate warning.

Explanation tree:

```
High Current + High Temperature
              |
       Excessive Load
              |
          Overheating
              |
      Equipment Damage Risk
              |
       Hazardous Condition
              |
         Send Warning
```

Learned rule:

IF current is high AND temperature is high, THEN classify the substation as hazardous and generate an alert.

Testing code:

```
cases = [
    (True, True),
    (True, False),
    (False, True),
    (False, False)
] * 5

def learned_rule(current, temperature):
    return current and temperature
```

```
for case in cases:
    print(case, "Hazardous:",
        learned_rule(case[0], case[1]))
```

Conclusion: EBL converts an explained example into a general rule that can be reused for future cases. It reduces repeated reasoning by storing the learned rule.

TASK 6 – Design Challenge: Reinforcement Learning for an Energy-Aware Smart Controller (13 Marks)

Stage 1 – Interpret the Problem

- **Input:** Room temperature, occupancy, outdoor temperature and electricity price.
- **Output:** Off, Low or High AC operation.
- **Objective:** Reduce electricity cost while maintaining comfortable room conditions.
- **Constraints:** The room should remain comfortable whenever people are present.
- **Episode:** One complete day of AC operation, divided into 15-minute intervals.
- **Termination:** The episode ends after the final time slot of the day.

Stage 2 – Propose the Strategy

Q-learning is selected because the controller must learn the best action through repeated interaction with the environment.

A fixed thermostat follows predefined rules, while supervised learning requires labelled examples of correct actions. Q-learning can learn through rewards and penalties without requiring labelled actions.

Stage 3 – Identify the Critical Information

State representation:

$$S = (Temperature, Occupancy, Price)$$

Actions:

$$A = \{Off, Low, High\}$$

Reward function:

$$R = -EnergyCost - ComfortPenalty$$

A higher penalty is given when the room becomes uncomfortable while occupied.

Exploration: Epsilon-greedy strategy is used to explore different actions.

It considers both immediate and future rewards.

Stage 4 – Design Complete Pseudocode

ALGORITHM QLearningController

BEGIN

Initialise Q-table with zeros.

Set learning rate alpha = 0.1.

Set discount factor gamma = 0.9.

Set exploration rate epsilon = 0.1.

FOR each episode DO

Initialise room state.

WHILE episode is not finished DO

Observe current state.

Select action using epsilon-greedy strategy.

Perform selected AC action.

Observe reward and next state.

Update Q-value:

Q(s,a) = Q(s,a) + alpha *

[reward + gamma * MAX Q(next_state)

- Q(s,a)]

Move to next state.

END WHILE

END FOR

Extract the best action for each state.

RETURN learned policy.

END

Stage 5 – Desk-Check

Consider three states:

- S1: Room occupied and hot.
- S2: Room occupied and comfortable.
- S3: Room unoccupied.

Actions: Low and High.

Assume learning rate = 0.5 and discount factor = 0.9.

Q-value update:

$$Q(s,a) = Q(s,a) + \alpha[r + \gamma \max Q(s',a') - Q(s,a)]$$

Step	State	Action	Reward	Next state	Updated Q-value
1	S1	High	5	S2	2.5
2	S2	Low	2	S3	1.0
3	S3	Off	3	S3	1.5
4	S1	Low	-4	S2	-2.0

For example, Step 1:

$$Q(S1, High) = 0 + 0.5(5) = 2.5$$

These values demonstrate how the controller updates its decisions using rewards.

Stage 6 – Implement, Instrument and Attack Your Own Design

Python implementation:

```
import numpy as np

np.random.seed(42)

states = 3

actions = 3

# Actions: 0 = Off, 1 = Low, 2 = High

Q = np.zeros((states, actions))

alpha = 0.1

gamma = 0.9

epsilon = 0.1

rewards = [

    [-5, 2, 5],

    [3, 2, -1],

    [4, 1, -2]

]

for episode in range(500):

    state = 0

    total_reward = 0

    for step in range(10):
```

```
        if np.random.random() < epsilon:
            action = np.random.randint(actions)
        else:
            action = np.argmax(Q[state])
        reward = rewards[state][action]
        next_state = np.random.randint(states)

        Q[state, action] += alpha * (
            reward + gamma * np.max(Q[next_state])
            - Q[state, action]
        )
        state = next_state
        total_reward += reward
    if episode % 50 == 0:
        print("Episode:", episode,
              "Reward:", total_reward)
print("\nLearned Q-table:")
print(Q)
print("\nFinal policy:")
print(np.argmax(Q, axis=1))
```

Expected interpretation: The Q-table gradually learns which AC action gives the highest expected reward for each state. The final policy selects the action with the highest Q-value.

Baseline comparison: A fixed-rule thermostat can be used as a baseline. Compare both methods using energy consumption, comfort violations and total reward over at least 20 random seeds.

Adversarial test:

Consider a situation where electricity prices are very high just before students enter the lecture hall. The controller may learn to keep the AC switched off to save energy. However, the room may become uncomfortable when students arrive.

This exposes a weakness in the reward function if energy savings receive more importance than occupant comfort.

Improvement: Increase the comfort-violation penalty and include upcoming occupancy information in the state representation.

Conclusion: Q-learning helps the smart controller learn energy-efficient AC operation through repeated interaction. A carefully designed reward function is important because the controller must balance electricity cost and human comfort.